

CIS 255 - Java Programming
Exam #3

Name: GEORGE WESTFALL

Date: 12/1/2025

1. (11 points) Given the declared and initialized LinkedList object below, display all the values in the list after each of the statement(s) is/are executed. If the list is empty, then write "empty". If the statement causes an error, then write "error". (Each statement builds on the one before)

List<String> list = new LinkedList<String>(); initial = empty

a.) list.add("Jim");

["Jim"]

g.) list.remove("Jim");

["Bill", "Karen", "Jim", "Sam"]

b.) list.add("Bill");

["Jim", "Bill"]

h.) list.clear();

empty

c.) list.add("Karen");

["Jim", "Bill", "Karen"]

i.) list.set(0, "Cindy");

error

d.) list.remove(3);

error

j.) List<String> fix =
 new LinkedList<String>();

fix.add("Adam");
list.addAll(fix);

["Adam"]

e.) list.add("Jim");

["Jim", "Bill", "Karen", "Jim"]

k.) list.add("Adam");

["Adam", "Adam"]

f.) list.add("Sam");

["Jim", "Bill", "Karen", "Jim", "Sam"]

2. (9 points) Given the declared and initialized LinkedHashMap object below, display all the keys and values in the map after each of the statement(s) is/are executed. If the map is empty, then write "empty". If the statement causes an error, then write "error". (Each statement builds on the one before)

```
Map<String, String> map = new LinkedHashMap<String, String>();
```

a.) map.put("AL", "Alabama");

{ "AL": "Alabama" }

f.) map.put("Tn", "Tennessee");

{ "Tn": "Tennessee" }

b.) map.put("NY", "New York");

{ "AL": "Alabama", "NY": "New York" }

g.) map.remove("AK");

{ "Tn": "Tennessee" }

c.) Map<String, String> fix1 =
new LinkedHashMap<String, String>();

```
fix1.put("HI", "Hawaii");  
fix1.put("TN", "Tennessee");  
map.putAll(fix1);
```

{ "AL": "Alabama", "NY": "New York",
"HI": "Hawaii", "TN": "Tennessee" }

h.) map.put("AK", "Alaska");

{ "Tn": "Tennessee",
"AK": "Alaska" }

i.) Map<String, String> fix2 = null;

```
fix2.put("AK", "Hawaii");  
fix2.put("TN", "Tennessee");  
map.putAll(fix2);
```

error

d.) map.remove("Al"); #1?

{ "AL": "Alabama", "NY": "New York",
"HI": "Hawaii", "TN": "Tennessee" }

e.) map.clear();

empty

3. Given the following Certification object class, answer the questions that follow.

```
package edu.calhoun.cis.java.intro.exam.exam3;

import java.util.Date;

public class Certification {
    private String certName = null;
    private double examLengthInHours = 0;
    private Date issueDate = null;

    public Certification() {
    }

    public Certification(String certName, double examLengthInHours, Date issueDate) {
        this.certName = certName;
        this.examLengthInHours = examLengthInHours;
        this.issueDate = issueDate;
    }

    public String getCertName() {
        return certName;
    }

    public void setCertName(String certName) {
        this.certName = certName;
    }

    public double getExamLengthInHours() {
        return examLengthInHours;
    }

    public void setExamLengthInHours(int examLengthInHours) {
        this.examLengthInHours = examLengthInHours;
    }

    public Date getIssueDate() {
        return issueDate;
    }

    public void setIssueDate(Date issueDate) {
        this.issueDate = issueDate;
    }

    public void showLevelsForDoD() {
        System.out.println("Not qualified to work for DoD");
    }

    @Override
    public String toString() {
        return "SecurityCertification [certName=" + certName
            + ", examLength=" + examLengthInHours
            + ", issueDate=" + issueDate + "]\n";
    }
}
```


- a. (10 points) Write a Java class `SecurityPlus` that that extends the `Certification` object.
- Ensure this class resides in the package `edu.calhoun.cis.java.intro.exam.exam3`.
 - Add the member variable `yearsExperience` as an integer.
 - Add getters and setters for the member variables.
 - Override the `showLevelsForDoD()` method to output "IAT Level II, IAM Level I".
 - The `SecurityPlus` object class should override the `toString()` method and display all variables (including the inherited ones).
- b. (10 points) Write a Java class `CISSP` that that extends the `Certification` object.
- Ensure this class resides in the package `edu.calhoun.cis.java.intro.exam.exam3`.
 - Add the member variables `yearsExperience` as an integer and `attendedBootcamp` as a boolean.
 - Add getters and setters for the member variables.
 - Override the `showLevelsForDoD()` method to output "IAT Level III, IAM Level III, IASAE II".
 - The `CISSP` object class should override the `toString()` method and display all variables (including the inherited ones).
- c. (10 points) Write the Java class `Exam3Q3` to test the `Certification`, `SecurityPlus`, and `CISSP` classes.
- Ensure class `Exam3Q3` resides in the package `edu.calhoun.cis.java.intro.exam.exam3`.
 - `Exam3Q3` should create 1 instance of the `Certification` object using the default constructor.
 - Output the created `Certification` object's values to the screen using a `System.out.println()`.
 - After displaying the `Certification` object's data, execute the `showLevelsForDoD()` method.
 - `Exam3Q3` should create 1 instance of the `SecurityPlus` object using the overloaded constructor.
 - Pass the following values to the constructor:
 - `"Security+"`
 - `1.5`
 - `new Date()`
 - `2`
 - Output the created `SecurityPlus` object's values to the screen using a `System.out.println()`.
 - After displaying the `Certification` object's data, execute the `showLevelsForDoD()` method.
 - `Exam3Q3` should create 1 instance of the `CISSP` object using the overloaded constructor.
 - Pass the following values to the constructor:
 - `"CISSP"`
 - `6.0`
 - `new Date()`
 - `11`
 - `true`
 - Output the created `CISSP` object's values to the screen using a `System.out.println()`.
 - After displaying the `Certification` object's data, execute the `showLevelsForDoD()` method.

B. a.

GEORGE WESTFAU

```
package edu.culhoun.cis.java.intro.exm. exam3;
import java.util.date;

public class SecurityPlus extends Certification {
    private int yearsExperience;

    public SecurityPlus (String certName, double examLengthInHours,
                        Date issueDate, int yearsExperience) {
        super(certName, examLengthInHours, issueDate);
        this.yearsExperience = yearsExperience;
    }

    public int getYearsExperience() {
        return yearsExperience;
    }

    public void setYearsExperience(int yearsExperience) {
        this.yearsExperience = yearsExperience;
    }

    void public void showLevelsForDoD() {
        System.out.println("IAT Level II, IAM Level I");
    }

    public String toString() {
        return super.toString() + ", yearsExperience = "
            + yearsExperience;
    }
}
```


36

GEORGE WESTFALL

```
package edu.caelhahn.cis.java.intro.exam.exam3;
```

```
import java.util.Date;
```

```
public class CISSP extends Certification {
```

```
    private int yearsExperience;
```

```
    private boolean attendedBootcamp;
```

```
    public CISSP(String var1, double var2, Date var3, int var4,  
                boolean var5) {
```

```
        super(var1, var2, var3);
```

```
        this.yearsExperience = var4;
```

```
        this.attendedBootcamp = var5;
```

```
    }
```

```
    public int getYearsExperience() {
```

```
        return yearsExperience;
```

```
    }
```

```
    public boolean hasAttendedBootcamp() {
```

```
        return attendedBootcamp;
```

```
    }
```

```
    public void setYearsExperience(int exp) {
```

```
        this.yearsExperience = exp;
```

```
    }
```

```
    public void setAttendedBootcamp(boolean ans) {
```

```
        this.attendedBootcamp = ans;
```

```
    }
```

```
    public void showLevelsForDoD() {
```

```
        System.out.println("IAT LEVEL III, IAM LEVEL VII,  
                            IASAE II");
```

```
    }
```

```
    public String toString() {
```

```
        return super.toString() + "yearsExperience = " +
```

```
        yearsExperience + "attendedBootcamp = " + attendedBootcamp
```

```
    };
```



```
package edu.culham.cis.java.intro.excm.excm3;
```

```
import java.util.Date;
```

```
public class Exam3Q3 {  
    public static void main (String[] args) {
```

```
        System.out.println ("--- 1. certification Test ---");
```

```
        Certification cent = new Certification ();
```

```
        System.out.println (cent);
```

```
        cent.showLevelsForDoD ();
```

```
        System.out.println ("2. SecurityPlus Object Test");
```

```
        SecurityPlus secPlus = new SecurityPlus ("security+",  
            1.5, new Date(), 2);
```

```
        System.out.println (secPlus);
```

```
        secPlus.showLevelsForDoD ();
```

```
        System.out.println ("3. CISSP Object Test");
```

```
        CISSP ciisp = new CISSP ("CISSP", 6.0, new Date(),  
            11, true);
```

```
        System.out.println (ciisp);
```

```
        ciisp.showLevelsForDoD ();
```

```
    }
```

```
}
```


4. Given the following Clearance interface, answer the questions that follow.

```
package edu.calhoun.cis.java.intro.exam.exam3;

public interface Clearance {
    public void showAcronym();
    public void showDescription();
    public void showImpactIfMadePublic();
}
```

a. (10 points) Write a Java class Confidential that implements the Clearance interface.

- Ensure class Confidential resides in the package `edu.calhoun.cis.java.intro.exam.exam3`.
- Add the private member variables for `acronym`, `description`, and `impactIfMadePublic`.
- Create the default constructor method which sets the member variables to the following values:
 - `acronym = "C"`
 - `description = "Confidential"`
 - `impactIfMadePublic = "Damage or be prejudicial to national security"`
- Implement the `showAcronym()`, `showDescription()`, and `showImpactIfMadePublic()` methods using a `System.out.println()` to output the appropriate variables to the screen.

b. (10 points) Write a Java class Secret that implements the Clearance interface.

- Ensure class Secret resides in the package `edu.calhoun.cis.java.intro.exam.exam3`.
- Add the private member variables for `acronym`, `description`, and `impactIfMadePublic`.
- Create the default constructor method which sets the member variables to the following values:
 - `acronym = "S"`
 - `description = "Secret"`
 - `impactIfMadePublic = "Serious damage to national security"`
- Implement the `showAcronym()`, `showDescription()`, and `showImpactIfMadePublic()` methods using a `System.out.println()` to output the appropriate variables to the screen.

c. (10 points) Write a Java class TopSecret that implements the Clearance interface.

- Ensure class `TopSecret` resides in the package `edu.calhoun.cis.java.intro.exam.exam3`.
- Add the private member variables for `acronym`, `description`, and `impactIfMadePublic`.
- Create the default constructor method which sets the member variables to the following values:
 - `acronym = "TS"`
 - `description = "Top Secret"`
 - `impactIfMadePublic = "Exceptionally grave damage to national security"`
- Implement the `showAcronym()`, `showDescription()`, and `showImpactIfMadePublic()` methods using a `System.out.println()` to output the appropriate variables to the screen.

d. (10 points) Write the Java class `Exam3Q4` to test the classes that implement the `Clearance` interface.

- Ensure class `Exam3Q4` resides in the package `edu.calhoun.cis.java.intro.exam.exam3`.
- `Exam3Q4` should create 1 `Confidential` object, 1 `Secret` object, and 1 `TopSecret` object.
- Each `Clearance` object after creation should be added to a `List` that will contain `Clearance` objects.
- Loop through the list and do the following at each iteration:
 - Execute the interface method `showAcronym()`
 - Execute the interface method `showDescription()`
 - Execute the interface method `showImpactIfMadePublic()`


```
package edu.calhoun.cis.java.intro.exercises.Exam3;
```

```
public class Confidential implements Clearance {
```

```
    private String acronym;
```

```
    private String description;
```

```
    private String impactIfMadePublic;
```

```
    public Confidential() {
```

```
        this.acronym = "C";
```

```
        this.description = "Confidential";
```

```
        this.impactIfMadePublic = "Damage or be  
        prejudicial to national security";
```

```
    }
```

```
    public void showAcronym() {
```

```
        System.out.println(this.acronym);
```

```
    }
```

```
    public void showDescription() {
```

```
        System.out.println(this.description);
```

```
    }
```

```
    public void showImpactIfMadePublic() {
```

```
        System.out.println(this.impactIfMadePublic);
```

```
    }
```


4b

George Westfall

```
package edu.cuhuwcis.java.intro.exer.exer3;
```

```
public class Secret implements Clearance {
```

```
    private String acronym;
```

```
    private String description;
```

```
    private String impactIfMadePublic;
```

```
    public Secret() {
```

```
        this.acronym = "S";
```

```
        this.description = "Secret";
```

```
        this.impactIfMadePublic = "Serious damage to national security";
```

```
    public void showAcronym() {
```

```
        System.out.println("Acronym: " + this.acronym);
```

```
    public void showDescription() {
```

```
        System.out.println("Description: " + this.description);
```

```
    public void showImpactIfMadePublic() {
```

```
        System.out.println("Impact if Made Public: " +
```

```
            this.impactIfMadePublic());
```


4c

George Washington

```
package edu.calhoun.cis.java.intro.exam.exam3;
```

```
public class TopSecret implements Clearance {
```

```
    private String acronym;
```

```
    private String description;
```

```
    private String impactIfMadePublic;
```

```
    public TopSecret () {
```

```
        this.acronym = "TS";
```

```
        this.description = "Top Secret";
```

```
        this.impactIfMadePublic = "Exceptionally grave  
damage to national security";
```

```
    public void showAcronym () {
```

```
        System.out.println("Acronym: " + this.acronym);
```

```
}
```

```
    public void showDescription () {
```

```
        System.out.println("Description: " +  
this.description);
```

```
}
```

```
public
```

```
    public void showImpactIfMadePublic () {
```

```
        System.out.println("Impact: " +  
this.impactIfMadePublic);
```

```
}
```

5. Given the following text file, ATMOSPHERES.DAT:

```
1 Troposphere 0 7
2 Stratosphere 7 31
3 Mesosphere 31 to 50
4 Thermosphere 50 440
```

And the following Atmosphere Java class, answer the questions that follow:

```
package edu.calhoun.cis.java.intro.exam.exam3;

public class Atmosphere {
    private String id = null;
    private String name = null;
    private String startHeightInMiles = null;
    private String endHeightInMiles = null;

    public Atmosphere() {
    }

    public Atmosphere(String id, String name,
        String startHeightInMiles, String endHeightInMiles) {
        this.id = id;
        this.name = name;
        this.startHeightInMiles = startHeightInMiles;
        this.endHeightInMiles = endHeightInMiles;
    }

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getStartHeightInMiles() {
        return startHeightInMiles;
    }

    public void setStartHeightInMiles(String startHeightInMiles) {
        this.startHeightInMiles = startHeightInMiles;
    }

    public String getEndHeightInMiles() {
        return endHeightInMiles;
    }

    public void setEndHeightInMiles(String endHeightInMiles) {
        this.endHeightInMiles = endHeightInMiles;
    }
}
```



```

@Override
public String toString() {
    return "Atmosphere [id=" + id
        + ", name=" + name
        + ", startHeightInMiles=" + startHeightInMiles
        + ", endHeightInMiles=" + endHeightInMiles + "]\n";
}
}

```

a. (10 points) Write the Java class Exam3Q5.

- Ensure class Exam3Q5 resides in the package edu.calhoun.cis.java.intro.exam.exam3.
- Create method readAtmospheres(list) in the Exam3Q5 class that reads the file ATMOSPHERES.DAT.
 - i. Each line in the data file will be stored into an Atmosphere Java class object and added to a list of Atmosphere objects.
- Once the file has been read and saved into a list of Atmosphere objects, create another method displayAtmospheres(list) which iterates the list and displays the information for each atmosphere.
- Add the method writeAtmosphere() to the Exam3Q5 class that writes the Atmosphere objects to a new data file ATMOSPHERES_NEW.DAT.
- Before writing the atmospheres to the new file, add a new Atmosphere object to the list by creating the method addAtmosphere(List<Atmosphere> atmospheres, String id, String name, String startHeightInMiles, String endHeightInMiles) and passing it the following values:

ID: 5

Name: Exosphere

Start Height in Miles: 440

End Height in Miles: 6200

- Show the output generated by executing this program.
- Outside of the program, copy and paste the contents of the newly created ATMOSPHERES_NEW.DAT to illustrate a successful write.

4d

George Westfall

```
package edu.cshuon.cis.java.intro.exam.exam3;

import java.util.ArrayList;
import java.util.List;

public class Exam3Q4 {
    public static void main(String[] args) {
        Confidential confidential = new Confidential();
        Secret secret = new Secret();
        Topsecret topsecret = new Topsecret();

        List<Clearance> clearanceList = new ArrayList<>();

        clearanceList.add(confidential);
        clearanceList.add(secret);
        clearanceList.add(topsecret);

        for (Clearance clearance : clearanceList) {
            clearance.showAcronym();
            clearance.showDescription();
            clearance.showImpact & Mode Public();
        }
    }
}
```


package edu. calhoun. cis. java. intro. exam. exam3;

```
import java.io. Buffer Read;  
import java.io. File;  
import java.io. File Input Stream;  
import java.io. File Writer;  
import java.io. Input Stream Reader;  
import java.io. Print Writer;  
import java.util. Array List;  
import java.util. List;  
import java.util. String Tokenizer;
```

```
public class Exam3Q5 {
```

```
public static void main (String[] args) {
```

```
List<Atmosphere> atmospheres = new ArrayList<>();
```

read Atmosphere (atmospheres):

add Atmosphere (atmospheres, "5", "Exosphere",
"440", "6200");

display Atmosphere (atmospheres);

5

```
public static void readAtmospheres(List<Atmosphere> list) {
```

FileInputStream flinput = null;
BufferedReader brReader = null;

try 5

```
File file = new File("ATMOSPHERES.DAT");
```

```
FileInput = new FileInputStream(file);  
brReader = new BufferedReader(new InputStreamReader  
                                (FileInput));
```


String line;

while ((line = br.readLine()) != null) {

StringTokenizer token = new StringTokenizer(line, " ");

String id = token.nextToken();

String name = token.nextToken();

String startHeight = token.nextToken();

String endHeight = token.nextToken();

Atmosphere atmosphere = new Atmosphere(id, name, startHeight, endHeight);

list.add(atmosphere);

} catch (Exception exp) {

{ finally {

flinput.close;

br.close;

}

}

public ~~void~~ static void displayAtmospheres(List<Atmosphere> list) {

for (Atmosphere atm : list) {

System.out.println(atm);

}

}

public static void addAtmosphere(List<Atmosphere> var1,
String var2, String var3, String var4, String var5) {

Atmosphere newATM = new Atmosphere(var1, var2, var3,
var4, var5);

atmospheres.add(newATM);

{


```
public static void writeAtmosphere (List<Atmosphere> list) {
```

```
    PrintWriter pwriter = null;
```

```
    try {
```

```
        pwriter = new PrintWriter (new FileWriter(
            "ATMOSPHERES_NEW.DAT"));
```

```
        for (Atmosphere atm: list) {
```

```
            pwriter.println(
```

```
                atm.getID() + " " +
```

```
                atm.getName() + " " +
```

```
                atm.getStartHeightInMiles() + " " +
```

```
                atm.getEndHeightInMiles());
```

```
        }
```

```
    } catch (Exception exp) {
```

```
        { finally {
```

```
            pwriter.close();
```

```
        }
```

```
    }
```


Sq - Screen Output

George Westfall

[id=1, name=Troposphere, startHeightInMiles=0, endHeightInMiles=7]
[id=2, name=Stratosphere, startHeightInMiles=7, endHeightInMiles=31]
[id=3, name=Mesosphere, startHeightInMiles=31, endHeightInMiles=50]
[id=4, name=Thermosphere, startHeightInMiles=50, endHeightInMiles=440]
[id=5, name=Exosphere, startHeightInMiles=440, endHeightInMiles=6200]

ATMOSPHERES.DAT FILE CONTENTS

1	troposphere	0	7
2	stratosphere	7	31
3	mesosphere	31	50
4	thermosphere	50	440
5	exosphere	440	6200